

CARTRIDGE & CLOUD · DOCUMENTACIÓN RC1

Cartridge & Cloud — Production Roadmap y Sprint Plan

Edición PDF con identidad visual unificada de VRM Games. El contenido procede íntegramente del archivo Markdown original.

PROYECTO

Cartridge & Cloud

DOCUMENTO

06 / 06_Production_Roadmap_y_Sprint_Plan.md

VERSIÓN

1.0

ESTADO

Fuente vigente de planificación y producción

AUTORÍA

VRM Games / Blas Luis Rocha González

FECHA DOCUMENTAL

2026-07-01

Control y metadatos del documento

Archivo fuente: 06_Production_Roadmap_y_Sprint_Plan.md
SHA-256 del Markdown: d00b156c0ad1c66069278119c55f76c738a577a3e2835f770208d1a5f2faadff
Tamaño del Markdown: 61,564 bytes
Conversión: representación visual completa; el Markdown original mantiene la autoridad sobre el texto fuente.

Front matter original

title	Cartridge & Cloud — Production Roadmap y Sprint Plan
author	VRM Games / Blas Luis Rocha González
date	2026-07-01
lang	es-ES
document_version	1.0
status	Fuente vigente de planificación y producción
project	Cartridge & Cloud
platform	PC / Steam
engine	Unity 6.3 LTS 6000.3.18f1
render_pipeline	URP 17.3.0
application_version_reference	0.0.17
documentary_baseline	v0.6

Índice

Cartridge & Cloud — Production Roadmap y Sprint Plan

- 0. Propósito del documento
 - 0.1. Jerarquía de autoridad
 - 0.2. Naturaleza de la fotografía de estado
 - 0.3. Estados de producción
- 1. Principios de producción
 - 1.1. Sprints cerrables
 - 1.2. Valor verificable
 - 1.3. Alcance congelado durante el sprint
 - 1.4. Calidad integrada
 - 1.5. Deuda explícita
 - 1.6. Producción individual sostenible
 - 1.7. Ninguna expansión antes del gate
- 2. Estado ejecutivo
 - 2.1. Resumen global
 - 2.2. Verdad funcional aceptada
 - 2.3. Riesgo activo principal
 - 2.4. Regla de continuidad
- 3. Fases del producto
- 4. Histórico cerrado: Sprints 0–15
 - 4.1. Regla de tratamiento histórico
 - 4.2. Ledger consolidado
 - 4.3. Herencia de regresión
 - 4.4. Hitos funcionales cubiertos
- 5. Sprint 16 — Art & Audio Representative Pass
 - 5.1. Estado
 - 5.2. Objetivo
 - 5.3. Trabajo ya aceptado dentro de Sprint 16
 - 5.4. Problemas no aceptados
 - 5.5. Decisión arquitectónica
 - 5.6. Jerarquía objetivo
 - 5.7. Plan de 11 pasos
 - 5.8. Dependencias de Sprint 16
 - 5.9. Fuera de alcance de Sprint 16
 - 5.10. Definition of Done de Sprint 16
 - 5.11. Condición de bloqueo
- 6. Sprint 17 — Vertical Slice Stabilization
 - 6.1. Estado
 - 6.2. Condición de apertura
 - 6.3. Objetivo
 - 6.4. Workstream A — Balance
 - 6.5. Workstream B — Rendimiento y profiling
 - 6.6. Workstream C — Persistencia y recuperación
 - 6.7. Workstream D — UX y accesibilidad
 - 6.8. Workstream E — Bug fixing
 - 6.9. Workstream F — Regresión completa
 - 6.10. Workstream G — Build Windows x64
 - 6.11. Workstream H — Documentación y aprobación
 - 6.12. Fuera de alcance de Sprint 17
 - 6.13. Definition of Done de Sprint 17
- 7. Gate H6 — Vertical Slice Approved
 - 7.1. Significado
 - 7.2. Evidencia obligatoria
 - 7.3. Decisiones posibles
 - 7.4. Consecuencia de PASS
- 8. Hitos de producto
 - 8.1. Regla de madurez de hitos
- 9. Roadmap posterior al vertical slice

- 9.1. Naturaleza provisional
- 9.2. Bloque PVS-0 — Freeze y revisión de producto
- 9.3. MVP-1 — Empleados
- 9.4. MVP-2 — Investigación y ampliaciones
- 9.5. MVP-3 — Puestos informáticos
- 9.6. H7 — MVP profundo de tienda
- 9.7. EXP-1 — Comercio online y logística
- 9.8. EXP-2 — Publishing
- 9.9. EXP-3 — Desarrollo interno
- 9.10. LATE-1 — Plataforma digital
- 9.11. LATE-2 — Infraestructura y servicios
- 9.12. LATE-3 — Mercado y competidores
- 10. Preparación de sprints post-H6
 - 10.1. No asignar números demasiado pronto
 - 10.2. Tamaño de sprint recomendado
 - 10.3. Spikes
- 11. Dependencias de producción
 - 11.1. Mapa general
 - 11.2. Dependencias por disciplina
 - 11.3. Dependencias bloqueantes
- 12. Backlog y prioridad
 - 12.1. Clases
 - 12.2. Regla de entrada
 - 12.3. Backlog no es roadmap
- 13. Definition of Ready
- 14. Definition of Done de sprint
- 15. Ciclo de vida de un sprint
 - 15.1. Preparación
 - 15.2. Ejecución
 - 15.3. Integración
 - 15.4. Validación
 - 15.5. Cierre
 - 15.6. Handoff
- 16. Gestión de alcance
 - 16.1. Cambio menor
 - 16.2. Cambio mayor
 - 16.3. Plantilla de cambio
 - 16.4. Protección contra sprints indefinidos
- 17. QA dentro del roadmap
 - 17.1. Pirámide
 - 17.2. Severidades
 - 17.3. Gate
 - 17.4. Regresión heredada
- 18. Build y versionado
 - 18.1. Versiones
 - 18.2. Tipos de build
 - 18.3. Registro mínimo
- 19. Evidencia obligatoria
 - 19.1. Por sprint
 - 19.2. Por hito
 - 19.3. Calidad de evidencia
- 20. Riesgos de producción
 - 20.1. Registro de riesgo
- 21. Métricas de producción
 - 21.1. Flujo
 - 21.2. Calidad
 - 21.3. Producto
 - 21.4. Interpretación
- 22. Revisión del roadmap
 - 22.1. Revisión semanal
 - 22.2. Revisión por sprint
 - 22.3. Revisión por hito
 - 22.4. Reestimación
- 23. Hitos comerciales
 - 23.1. Vertical Slice
 - 23.2. MVP de tienda

- 23.3. Demo pública
- 23.4. Alpha
- 23.5. Beta
- 23.6. Release Candidate
- 23.7. Lanzamiento
- 24. Elementos no comprometidos
- 25. Secuencia inmediata recomendada
- 26. Matriz de decisiones de fase

- 27. Plantilla de Sprint Charter
- 28. Plantilla de cierre
- 29. Regla de mantenimiento del roadmap
- Anexo A. Trazabilidad de fuentes
 - A.1. Uso de las fuentes
- Anexo B. Decisiones sustituidas
- Anexo C. Glosario de producción
- Anexo D. Ruta de almacenamiento

Cartridge & Cloud — Production Roadmap y Sprint Plan

0. Propósito del documento

Este documento transforma la visión de producto, el diseño de juego, la especificación del vertical slice, la arquitectura técnica, el modelo de datos y los recorridos UX en un plan de producción ordenado, verificable y gobernable.

Su función es responder:

- qué trabajo está realmente cerrado;
- qué trabajo permanece activo;
- qué condiciones permiten abrir el siguiente sprint;
- qué dependencias existen entre código, datos, escenas, contenido, UX, audio y QA;
- qué evidencia debe producir cada sprint;
- qué capacidades quedan fuera del alcance inmediato;
- cómo se evita que el proyecto crezca por acumulación descontrolada;
- qué hitos deben alcanzarse antes de demo, Alpha, Beta, Release Candidate y lanzamiento;
- cómo se revisa el roadmap cuando la realidad contradice las estimaciones.

Este archivo no es un calendario contractual. No asigna fechas ficticias a trabajo cuya capacidad y coste todavía no han sido medidos. Define **orden, gates, resultados, dependencias y condiciones de cierre**. Las fechas solo deberán incorporarse cuando exista:

1. una capacidad de producción observada;
2. un backlog preparado;
3. dependencias resueltas;
4. un riesgo asumible;
5. un alcance congelado;
6. criterios de aceptación verificables.

0.1. Jerarquía de autoridad

La consolidación aplica el siguiente orden:

1. 00_Enfoque_y_Alcance.md
2. 01_Game_Design_Document.md
3. 02_Vertical_Slice_Specification.md
4. 03_Technical_Design_Document.md
5. 04_Modelo_de_Datos.md
6. 05_UX_Flow.md
7. Production Roadmap / Sprint Plan v0.6
8. Sprint 16 Current Status
9. StoreInitial Authoring Plan
10. ADR-0035 – StoreInitial Manual Scene Authoring
11. Sprint 17 Opening Brief
12. Sprints 00–15 Consolidated Closure Ledger
13. Current Project Baseline Record
14. Current Project Handoff
15. Production Roadmap / Sprint Plan v0.5
16. Production Roadmap / Sprint Plan v0.4
17. Production Roadmap / Sprint Plan v0.3

Cuando exista una contradicción:

- prevalece el documento consolidado de mayor autoridad;
- el estado operativo demostrado prevalece sobre planes históricos;
- un sprint cerrado no se reabre para introducir mejoras no bloqueantes;
- una incidencia estructural descubierta después del cierre se registra como regresión o deuda;
- Sprint 16 no se cierra únicamente por pruebas automatizadas verdes;
- Sprint 17 no se abre antes de la aprobación visual y build posterior de Sprint 16;
- ningún sistema mayor nuevo entra en Sprint 17;
- las fases posteriores al vertical slice son propuestas condicionadas, no compromisos;
- una descripción futura detallada no equivale a backlog preparado;
- los hitos comerciales no se anuncian externamente sin validación de alcance y capacidad.

0.2. Naturaleza de la fotografía de estado

Los datos de estado contenidos en este documento corresponden a la baseline documental del **1 de julio de 2026**:

- versión de aplicación de referencia: 0.0.17;
- último commit técnico validado en la baseline: 091090c43855b0b26b09abe9335d18b978ac7eab;
- main observado por esa baseline: d54316c771aab2143993e99b9fd58f2f88016568;
- Sprints 0-15: CLOSED / PASS;
- Sprint 16: IN PROGRESS;
- Sprint 17: PENDING;
- baseline automatizada: 1215 EditMode + 70 PlayMode = 1285 PASS.

Estos SHA y resultados deben considerarse **trazabilidad histórica de la baseline**, no una afirmación permanente sobre el repositorio. Antes de publicar una nueva baseline se deberá registrar el SHA real validado, la versión de aplicación y los resultados de pruebas correspondientes.

0.3. Estados de producción

Estado	Significado
PROPOSED	Idea o bloque candidato todavía no aprobado.
DISCOVERY	Trabajo de investigación limitado para reducir incertidumbre.
READY	Cumple Definition of Ready y puede abrirse.
IN PROGRESS	Sprint activo con alcance congelado.
BLOCKED	Existe un impedimento externo o técnico que impide progresar.
IN REVIEW	Implementación terminada, pendiente de QA, aceptación o evidencia.
CLOSED / PASS	Cumple Definition of Done y su cierre está registrado.
CLOSED / ACCEPTED DEBT	Cerrado con deuda explícitamente aceptada y planificada.
DEFERRED	Aplazado sin considerarse cancelado.
CANCELLED	Eliminado del roadmap por decisión registrada.
SUPERSEDED	Sustituido por una decisión posterior.

Solo debe existir un sprint principal **IN PROGRESS** por línea de producción individual, salvo que un spike aislado haya sido aprobado y no compita por la misma capacidad.

1. Principios de producción

1.1. Sprints cerrables

Cada sprint debe producir un resultado que pueda demostrarse, probarse y aceptar o rechazar.

Un sprint no debe definirse como:

- “seguir trabajando en la tienda”;
- “mejorar la UI”;
- “hacer arte”;
- “corregir bugs” sin un conjunto delimitado;
- “avanzar en empleados”;
- “pulir el juego”.

Debe definirse como una capacidad o gate concreto, por ejemplo:

- autorar y conectar `StoreInitial.unity`;
- validar una transferencia de inventario atómica;
- cerrar el Golden Path en build Windows x64;
- implementar contratación y asignación de una única función de empleado;
- validar una migración de schema concreta.

1.2. Valor verificable

Cada sprint entrega al menos uno de estos tipos de valor:

- capacidad jugable;
- reducción medible de riesgo;
- integración de sistemas;

- contenido representativo;
- estabilidad;
- rendimiento;
- accesibilidad;
- persistencia;
- herramienta de producción;
- evidencia de calidad;
- cierre documental necesario para continuar.

La actividad no equivale a progreso. Los archivos modificados, horas invertidas o volumen de código no sustituyen al resultado aceptado.

1.3. Alcance congelado durante el sprint

Al abrir un sprint:

- se congela el objetivo;
- se congelan los criterios de aceptación;
- se declara la versión objetivo;
- se enumeran dependencias y exclusiones;
- se identifica la evidencia exigida;
- se define qué ocurre si una dependencia falla.

Un cambio de alcance durante el sprint solo puede:

1. corregir un defecto de definición;
2. retirar trabajo para proteger el objetivo;
3. introducir una corrección bloqueante imprescindible;
4. responder a una decisión formal de seguridad, datos o compatibilidad.

Añadir una capacidad atractiva pero no necesaria requiere aplazar otra de esfuerzo comparable o cerrar el sprint antes de abrir uno nuevo.

1.4. Calidad integrada

QA, documentación y trazabilidad forman parte del trabajo, no una fase opcional posterior.

Cada sprint debe incluir:

- pruebas unitarias o de integración cuando correspondan;
- recorrido manual;
- actualización de matrices;
- registro de defectos;
- revisión de persistencia;
- revisión de build si cambia el flujo de producción;
- documentación;
- evidencia reproducible.

1.5. Deuda explícita

Una deuda solo puede aceptarse si se registra:

- causa;
- impacto;
- riesgo;
- workaround;
- propietario;
- condición de pago;
- prioridad;
- sprint o gate candidato.

La deuda no registrada se considera incertidumbre, no planificación.

1.6. Producción individual sostenible

El roadmap asume una producción principalmente individual.

Por tanto:

- se prioriza reutilización;
- se limita contenido antes de validar sistemas;
- se evitan pipelines que exijan mantenimiento continuo desproporcionado;
- se automatizan comprobaciones repetibles;

- se documenta para reducir dependencia de memoria;
- se protege tiempo de integración y QA;
- se evita abrir varias líneas de gameplay mayores a la vez;
- se rechazan fechas basadas en velocidad aspiracional.

1.7. Ninguna expansión antes del gate

No se inicia producción funcional de empleados, investigación, puestos informáticos, comercio online, publishing, desarrollo interno, plataforma, infraestructura o mercado avanzado antes de aprobar el vertical slice.

Se permiten únicamente:

- notas conceptuales;
- decisiones de compatibilidad arquitectónica;
- spikes estrictamente limitados;
- pruebas de importación;
- investigación de riesgo;
- prototipos desechables con aprobación explícita.

2. Estado ejecutivo

2.1. Resumen global

Bloque	Estado	Resultado
Sprints 0–15	CLOSED / PASS	vertical slice funcional integrado
Sprint 16	IN PROGRESS	escena, arte, audio y presentación representativa
Sprint 17	PENDING	estabilización, balance, rendimiento, QA y build
H0–H5	completados funcionalmente	fundación, control, inventario, ventas, día y persistencia
H6	pendiente	aprobación formal del vertical slice
Post-H6	no aprobado	roadmap condicionado a revisión posterior

2.2. Verdad funcional aceptada

La baseline considera funcionales e integrados:

- fundación Unity/URP;
- repositorio y assemblies;
- Bootstrap y flujo de escenas;
- sesión, IDs y slots;
- movimiento y cámara;
- grid y placement;
- shell técnico de tienda;
- productos e inventario;
- proveedores, pedidos y recepción;
- displays y reposición;
- perfiles y spawning de clientes;
- shopping y reservas lógicas;
- cola y checkout;
- ciclo diario;
- economía y resultados;
- persistencia integrada, backup y recuperación;
- UI/UX del vertical slice.

Esta aceptación no implica que la presentación visual final esté aprobada.

2.3. Riesgo activo principal

El riesgo actual no es la ausencia del bucle funcional. Es la transición desde una integración técnica y procedural hacia una escena representativa, autorada y revisable sin romper:

- grid;
- navegación;
- inventario;
- displays;
- clientes;
- checkout;

- jornada;
- persistencia;
- UI;
- build.

2.4. Regla de continuidad

El próximo trabajo debe comenzar donde termina la evidencia aceptada.

No debe:

- reconstruir Sprints 0–15;
- reabrir sistemas cerrados para refactorizaciones cosméticas;
- sustituir arquitectura estable sin un problema demostrado;
- eliminar fallbacks antes de validar la migración;
- cambiar Build Profiles antes de conectar `StoreInitial`;
- depender de nombres de `GameObject` para registrar sistemas;
- cerrar Sprint 16 solo porque los tests estén verdes.

3. Fases del producto

Código	Fase	Resultado requerido
R0	Fundación documental y técnica	proyecto reproducible y gobernado
P1	Mundo y control	entrada, navegación, jugador y cámara
P2	Construcción	grid, placement, ocupación y acceso
P3	Productos y logística	stock, pedidos, recepción, displays
P4	Clientes y ventas	agentes, reservas, cola y checkout
P5	Día, economía y save	bucle cerrado y persistente
VS	Vertical Slice	experiencia representativa, estable y validada
MVP	Profundidad de tienda	empleados, investigación y puestos

Código	Fase	Resultado requerido
EXP	Expansión empresarial	comercio online, publishing y desarrollo
LATE	Ecosistema avanzado	plataforma, infraestructura y mercado
REL	Lanzamiento	Alpha, Beta, RC, Steam y soporte inicial

Las fases **R0–P5** están funcionalmente cerradas mediante Sprints 0–15. La fase **VS** permanece abierta hasta cerrar Sprints 16 y 17.

4. Histórico cerrado: Sprints 0–15

4.1. Regla de tratamiento histórico

Los Sprints 0–15 se consideran **CLOSED / PASS**.

Sus registros detallados permanecen como evidencia, pero el desarrollo activo debe utilizar el resultado consolidado. No es necesario releer cada charter histórico para continuar Sprint 16.

Un defecto descubierto en una capacidad cerrada se gestiona como:

- regresión bloqueante dentro del sprint actual, si impide su objetivo;
- incidencia del backlog, si no bloquea;
- deuda técnica, si la capacidad funciona pero presenta coste futuro;
- cambio de diseño, si la regla original deja de ser válida.

4.2. Ledger consolidado

Sprint	Nombre	Versión	Resultado aceptado
0	Project Foundation	0.0.1	Unity/URP, repositorio, assemblies, escenas base, pruebas y build
1	Bootstrap & Scene Flow	0.0.2	ApplicationRoot persistente y navegación Bootstrap/MainMenu/Store
2	Core Data & Save Skeleton	0.0.3	IDs, sesión, tres slots y snapshot mínimo versionado
3	Player Movement & Camera	0.0.4	contextos de input, click-to-move, órbita y zoom

Sprint	Nombre	Versión	Resultado aceptado
4	Grid & Placement Foundation	0.0.5	grid 0,5 m , huellas, preview, rotación, ocupación y retirada
5	Store Shell & Access Validation	0.0.6	tienda 10 × 15 m , grid 20 × 30 , entrada y validación de rutas
6	Product & Inventory Core	0.0.7	definiciones, cantidades, contenedores y transferencias atómicas
7	Supplier Orders & Receiving	0.0.8	proveedores, órdenes, entregas, cajas y recepción
8	Displays & Restocking	0.0.9	displays, asignación, capacidad, stock visible y reposición
9	Customer Profiles & Spawning	0.0.10	perfiles, spawning determinista, navegación y paciencia
10	Shopping & Reservations	0.0.11	intención, reservas lógicas, carrito y protección de stock
11	Queue & Checkout	0.0.12	cola FIFO, estación y checkout autoritativo
12	Day Cycle & Store Closure	0.0.13	BeforeOpen/Open/Closing/Closed y tiempo lógico
13	Economy & Reports	0.0.14	dinero exacto, costes, precios, ledger y resultados
14	Save/Load Complete Slice	0.0.15	snapshot integrado, backup, recuperación, reparación y schema
15	UI/UX Integration	0.0.16	slots, HUD, Operations, tutorial y accesibilidad base

4.3. Herencia de regresión

La baseline conjunta aceptada es:

- 1215 pruebas EditMode;
- 70 pruebas PlayMode;
- 1285 pruebas totales en PASS.

Esta cifra:

- no sustituye la validación manual;
- no garantiza aceptación visual;
- no garantiza que la escena representativa esté en build;
- no autoriza a eliminar fallbacks;
- debe volver a ejecutarse después de cambios relevantes.

4.4. Hitos funcionales cubiertos

Hito	Estado	Evidencia funcional
H0	completado	proyecto compila y genera build
H1	completado	el jugador se mueve y puede construir
H2	completado	inventario y recepción sin pérdida
H3	completado	un cliente puede completar una compra
H4	completado	un día puede abrirse, operar y cerrarse
H5	completado	una semana puede guardarse y cargarse
H6	pendiente	vertical slice representativo aprobado

5. Sprint 16 — Art & Audio Representative Pass

5.1. Estado

IN PROGRESS

Versión de aplicación de referencia: 0.0.17.

5.2. Objetivo

Sustituir la presentación provisional del vertical slice por una tienda inicial representativa, autorada y coherente, preservando la funcionalidad cerrada en Sprints 0-15.

Sprint 16 no es un sprint de nuevos sistemas de negocio. Su objetivo es demostrar que la capa representativa puede integrarse sin romper la verdad técnica.

5.3. Trabajo ya aceptado dentro de Sprint 16

La documentación vigente registra como completados o validados:

- Fase 1 funcional;

- placeholders representativos;
- audio y VFX base;
- catálogos;
- integration tests;
- Golden Path previo;
- build previa a la integración representativa;
- importación de assets representativos;
- prefabs;
- transferencia de LOD;
- conexión con catálogo runtime;
- suites EditMode y PlayMode en PASS.

Estos resultados no cierran el sprint porque la composición visual actual no está aprobada.

5.4. Problemas no aceptados

Permanecen abiertos:

- muros;
- puerta;
- mobiliario de almacén;
- composición representativa;
- conexión definitiva de `StoreInitial`;
- desactivación controlada del shell procedural;
- supresión del clic de mundo sobre UI;
- validación manual posterior;
- Golden Path posterior;
- build posterior a integración.

5.5. Decisión arquitectónica

ADR-0035 establece:

- crear `StoreInitial.unity`;

- crear `StoreInitialEnvironment.prefab`;
- autorar arquitectura y mobiliario fijo manualmente;
- resolver referencias mediante `StoreInitialSceneContext`;
- permitir que runtime cree únicamente estado dinámico;
- mantener colliders y grid técnicos como autoridad;
- conservar temporalmente el shell procedural como fallback;
- modificar la lista de escenas de build solo después de conectar runtime.

5.6. Jerarquía objetivo

```
StoreInitialEnvironment
├─ Architecture
├─ InitialFurniture
├─ Lighting
├─ Anchors
└─ TechnicalColliders
```

No deben incluirse dentro del prefab del entorno:

- `EventSystem`;
- HUD;
- cámara;
- managers globales;
- servicios de guardado;
- `ApplicationRoot`.

5.7. Plan de 11 pasos

Paso 1 — Crear `StoreInitial.unity`

Objetivo: partir de una copia validada de la escena Store funcional.

Entregables:

- escena creada;
- referencias conservadas;

- ejecución funcional sin cambios visuales;
- diferencia de escena revisable.

Aceptación:

- abre sin errores;
- movimiento, cámara, UI y sistemas funcionan;
- suites relevantes continúan verdes;
- no se elimina todavía la escena anterior.

Paso 2 — Crear `StoreInitialEnvironment.prefab`

Objetivo: encapsular únicamente el contenido fijo del entorno.

Entregables:

- prefab;
- jerarquía limpia;
- ownership explícito;
- ausencia de managers y servicios globales.

Aceptación:

- puede instanciarse sin duplicar sistemas;
- las referencias externas son mínimas y explícitas;
- no depende de nombres accidentales.

Paso 3 — Montar suelo y arquitectura

Objetivo: autorar manualmente una tienda legible de aproximadamente `10 × 15 m`.

Entregables:

- suelo;
- muros;
- acceso;
- zonas;
- colliders;
- relación coherente con grid `20 × 30`.

Aceptación:

- no existen huecos o solapes visuales graves;
- el jugador comprende entrada, venta, exposición, almacén y recepción;
- la arquitectura no bloquea rutas válidas;
- se mantiene la escala aprobada.

Paso 4 — Configurar puerta

Objetivo: integrar la puerta como elemento visual y funcional.

Entregables:

- hojas y marco;
- collider o trigger;
- animación o comportamiento;
- referencia funcional.

Aceptación:

- no bloquea entrada;
- su estado visual coincide con el estado lógico;
- no depende de un nombre de GameObject;
- funciona durante apertura y cierre.

Paso 5 — Colocar mobiliario inicial

Objetivo: crear una composición representativa y utilizable.

Entregables:

- mostrador;
- exposición;
- almacén;
- recepción;
- espacio de circulación;
- familias de mobiliario representativas.

Aceptación:

- no se invalida la ocupación técnica;
- los puntos de interacción son accesibles;
- el layout permite el Golden Path;
- la tienda no parece vacía ni congestionada.

Paso 6 — Añadir anchors y roots técnicos

Objetivo: separar referencias funcionales de la jerarquía visual.

Entregables:

- anchors;
- roots;
- zonas;
- colliders técnicos;
- referencias documentadas.

Aceptación:

- los sistemas pueden resolver entradas, salidas, checkout, recepción y displays;
- cambiar un modelo visual no cambia un ID funcional;
- no se utilizan búsquedas globales frágiles.

Paso 7 — Crear `StoreInitialSceneContext`

Objetivo: declarar la composición requerida por runtime.

Entregables:

- componente de contexto;
- referencias serializadas;
- validación de referencias obligatorias;
- mensajes de diagnóstico.

Aceptación:

- el contexto falla de forma controlada;
- las referencias faltantes se detectan antes de mutar;
- el código no deduce arquitectura desde bounds o nombres.

Paso 8 — Conectar runtime

Objetivo: operar la escena autorada con los sistemas existentes.

Entregables:

- composición;
- registro;
- restauración;
- conexión con catálogo;
- conexión de UI.

Aceptación:

- nueva partida;
- carga;
- placement;
- inventario;
- clientes;
- checkout;
- jornada;
- economía;
- guardado y carga funcionan en `StoreInitial`.

Paso 9 — Desactivar generación procedural del shell

Objetivo: evitar duplicación visual sin perder fallback antes de validar.

Entregables:

- flag o ruta de composición;
- procedural desactivado en `StoreInitial`;
- fallback conservado de forma aislada;
- plan de retirada posterior.

Aceptación:

- no aparece doble arquitectura;
- el fallback no se ejecuta accidentalmente;

- la escena antigua continúa disponible hasta aprobar la migración.

Paso 10 — Corregir supresión de clic sobre UI

Objetivo: impedir que la UI y el mundo procesen el mismo clic.

Entregables:

- comprobación de `EventSystem`;
- contextos de input correctos;
- pruebas;
- recorrido manual.

Aceptación:

- operar HUD u Operations no mueve al jugador;
- cerrar paneles no confirma placement;
- no se generan acciones de mundo bajo modales;
- teclado, ratón y Escape respetan prioridad.

Paso 11 — Validar tests, Golden Path y build

Objetivo: cerrar la migración con evidencia reproducible.

Entregables:

- `EditMode`;
- `PlayMode`;
- validación manual;
- Golden Path;
- build Windows x64;
- `Player.log`;
- capturas;
- cierre documental.

Aceptación:

- suites en PASS;
- Golden Path completado;

- presentación aprobada;
- build arranca por Bootstrap;
- `StoreInitial` aparece en el flujo correcto;
- no existen S0/S1;
- no hay errores bloqueantes en `Player.log`.

5.8. Dependencias de Sprint 16

Dependencia	Tipo	Condición
Store funcional	técnica	conservar comportamiento durante duplicación
assets representativos	contenido	importados, prefabs válidos y escala revisada
catálogo runtime	datos	referencias estables
grid y colliders	técnica	siguen siendo autoridad
UI/UX S15	funcional	no debe romperse
persistencia S14	funcional	debe restaurar estado integrado
tests 0-15	QA	herencia de regresión
build profile	producción	no cambiar antes de conectar escena

5.9. Fuera de alcance de Sprint 16

- nuevos sistemas de empleados;
- investigación;
- puestos informáticos;
- comercio online;
- rediseño de economía;
- expansión masiva de catálogo;
- reescritura del save;
- sustitución general de arquitectura;
- eliminación prematura de fallbacks;
- arte final completo;
- campaña comercial;

- Steamworks.

5.10. Definition of Done de Sprint 16

Sprint 16 se cierra cuando:

1. `StoreInitial.unity` existe y está autorada;
2. `StoreInitialEnvironment.prefab` contiene solo entorno fijo;
3. arquitectura, puerta y mobiliario están aprobados visualmente;
4. `StoreInitialSceneContext` resuelve referencias explícitas;
5. runtime opera la escena;
6. el shell procedural está desactivado en la ruta nueva;
7. el fallback permanece aislado hasta confirmar migración;
8. el clic sobre UI no produce acciones de mundo;
9. movimiento, cámara, build mode, Operations y pausa funcionan;
10. inventario, displays, clientes, checkout, día y economía funcionan;
11. guardado y carga restauran estado;
12. EditMode y PlayMode pasan;
13. Golden Path pasa;
14. la build posterior a integración pasa;
15. `Player.log` no contiene errores bloqueantes;
16. la evidencia está archivada;
17. la documentación y trazabilidad están actualizadas;
18. existe aprobación manual visual explícita.

5.11. Condición de bloqueo

Si la composición representativa exige cambiar una regla funcional cerrada, el sprint debe detenerse y clasificar el problema como:

- defecto de asset;
- defecto de autoría;
- defecto técnico;
- deuda de arquitectura;

- cambio de diseño;
- riesgo de rendimiento.

No debe resolverse ocultando el problema mediante offsets arbitrarios o búsquedas por nombre.

6. Sprint 17 — Vertical Slice Stabilization

6.1. Estado

PENDING

6.2. Condición de apertura

Sprint 17 se abre únicamente después de:

- aprobación visual manual de Sprint 16;
- conexión funcional de `StoreInitial`;
- Golden Path post-integración;
- build post-integración;
- registro de SHA y versión;
- cierre documental de Sprint 16;
- lista inicial de defectos clasificada.

Si una de estas condiciones falta, Sprint 17 permanece cerrado.

6.3. Objetivo

Convertir el vertical slice integrado y representativo en una baseline estable, medible, reproducible y aprobable.

Sprint 17 no debe añadir sistemas mayores.

6.4. Workstream A — Balance

Alcance:

- precios;
- costes;
- stock inicial;
- tiempos;
- paciencia;
- frecuencia de clientes;
- ritmo de pedidos;
- capacidad de displays;
- duración de jornada;
- liquidez;
- resultados diarios.

Proceso:

1. definir configuración de prueba;
2. ejecutar varias jornadas;
3. registrar métricas;
4. identificar extremos;
5. ajustar datos, no código, cuando sea posible;
6. repetir;
7. documentar valores y razones.

Aceptación:

- el Golden Path es completables sin grind artificial;
- no existe fracaso inevitable por valores iniciales;
- las decisiones producen diferencias observables;
- los valores viven en configuración cuando corresponde;
- el balance no oculta defectos lógicos.

6.5. Workstream B — Rendimiento y profiling

Medir:

- FPS;
- frame time;
- CPU;
- GPU;
- memoria;
- allocations;
- carga de escena;
- guardado;
- carga;
- spawning;
- navegación;
- UI;
- instanciación de stock;
- VFX y audio.

Reglas:

- medir en build;
- registrar hardware y configuración;
- evitar optimización especulativa;
- priorizar picos reproducibles;
- no sacrificar claridad sin evidencia;
- verificar que la optimización no rompe determinismo.

Aceptación:

- objetivos de rendimiento definidos y registrados;
- no existen allocations recurrentes graves en rutas críticas aceptadas;
- cargas y guardados no producen bloqueos inaceptables;
- la escena representativa no introduce regresión significativa;
- los cambios tienen comparación antes/después.

6.6. Workstream C — Persistencia y recuperación

Validar:

- tres slots;
- nueva partida;
- guardado;
- carga;
- reemplazo;
- borrado;
- backup;
- recuperación;
- schema actual;
- reparación;
- estado de placement;
- inventario;
- reservas;
- checkout;
- día;
- economía;
- UI.

Aceptación:

- ninguna ruta destruye un guardado válido;
- el primario inválido no sobrescribe el backup;
- la recuperación se comunica;
- guardar durante un punto permitido es atómico;
- cargar produce equivalencia observable;
- los errores se registran sin exponer detalles inapropiados al usuario.

6.7. Workstream D — UX y accesibilidad

Revisar:

- jerarquía del HUD;
- navegación de Operations;
- input;
- Escape;
- modales;
- estados vacíos;
- mensajes de error;
- escala de UI;
- contraste;
- feedback no dependiente solo del color;
- velocidad de cámara;
- avisos visuales;
- localización ES/EN;
- truncamientos;
- tutorial;
- resultados diarios.

Aceptación:

- Golden Path se comprende sin explicación externa;
- la UI no mueve al jugador;
- no existen bloqueos de foco;
- los mensajes indican causa y siguiente acción;
- los estados críticos tienen alternativa no cromática;
- no existen truncamientos críticos en ES/EN;
- las opciones base persisten.

6.8. Workstream E — Bug fixing

Prioridad:

1. S0;
2. S1;
3. S2 que amenace el gate;

4. regresiones;
5. S3 con alto impacto UX;
6. S4 solo si no desplaza trabajo obligatorio.

Reglas:

- cada defecto tiene reproducción;
- cada corrección tiene verificación;
- no mezclar refactor amplio con fix pequeño;
- actualizar pruebas cuando sea razonable;
- registrar riesgo de regresión;
- reabrir el recorrido afectado.

6.9. Workstream F — Regresión completa

Debe incluir:

- EditMode;
- PlayMode;
- pruebas de integración;
- Golden Path;
- smoke tests de escenas;
- nueva partida;
- carga;
- build;
- cierre de jornada;
- recuperación;
- input UI/mundo.

La cifra histórica de **1285 PASS** es referencia mínima, no garantía de que el total futuro sea idéntico.

6.10. Workstream G — Build Windows x64

Preflight:

- versión;

- SHA;
- escenas;
- Bootstrap primero;
- `StoreInitial` conectada;
- configuración;
- símbolos de desarrollo según objetivo;
- carpeta limpia;
- dependencias;
- espacio disponible.

Validación:

- arranque;
- MainMenu;
- slots;
- Golden Path;
- guardado;
- carga;
- salida;
- `Player.log`;
- no depender del Editor;
- registrar checksum o identificador de build.

6.11. Workstream H — Documentación y aprobación

Actualizar:

- estado de proyecto;
- roadmap;
- vertical slice specification;
- QA plan;
- QA matrix;
- known issues;
- build record;

- traceability;
- changelog;
- handoff;
- baseline.

6.12. Fuera de alcance de Sprint 17

- empleados;
- investigación;
- puestos informáticos;
- comercio online;
- publishing;
- desarrollo interno;
- plataforma;
- infraestructura;
- nuevos grandes catálogos;
- rediseño artístico completo;
- nuevas escenas de negocio;
- Steamworks;
- demo pública;
- reescritura de arquitectura sin defecto bloqueante.

6.13. Definition of Done de Sprint 17

1. Sprint 16 está cerrado.
2. Balance de referencia registrado.
3. Profiling ejecutado en build.
4. Regresiones de rendimiento críticas corregidas.
5. Persistencia y recuperación pasan.
6. UX y accesibilidad base pasan.
7. ES/EN no presentan defectos críticos.
8. No existen S0/S1.
9. S2 restantes están cerrados o aceptados formalmente.

10. Suite completa pasa.
11. Golden Path pasa en build.
12. Build Windows x64 está identificada.
13. `Player.log` está revisado.
14. Defectos y known issues están actualizados.
15. Evidencias están archivadas.
16. Documentación está sincronizada.
17. Se celebra revisión de H6.
18. Existe decisión formal `PASS`, `PASS WITH ACCEPTED DEBT` o `FAIL`.

7. Gate H6 — Vertical Slice Approved

7.1. Significado

H6 no significa “todos los sistemas soñados están implementados”.

Significa que existe una porción representativa del juego que demuestra:

- fantasía central;
- arquitectura;
- bucle comercial;
- calidad de interacción;
- consistencia de datos;
- persistencia;
- presentación;
- rendimiento;
- capacidad de producción.

7.2. Evidencia obligatoria

- build identificada;

- SHA;
- versión;
- configuración de hardware;
- suites;
- Golden Path;
- capturas o vídeo;
- `Player.log`;
- resultados de rendimiento;
- resultados de persistencia;
- matriz de aceptación;
- lista de defectos;
- known issues;
- decisiones de deuda;
- baseline documental.

7.3. Decisiones posibles

PASS

Todos los criterios obligatorios están cumplidos.

PASS WITH ACCEPTED DEBT

No existen S0/S1 y la deuda restante:

- no invalida la fantasía;
- no amenaza datos;
- no impide build;
- tiene owner;
- tiene plan;
- está aceptada formalmente.

FAIL

Existe cualquier bloqueo que impida:

- completar Golden Path;
- confiar en guardado;
- reproducir build;
- operar la tienda;
- mantener rendimiento mínimo;
- comprender el flujo;
- aceptar visualmente la escena.

7.4. Consecuencia de PASS

Después de H6:

1. congelar baseline;
2. archivar evidencia;
3. celebrar retrospectiva;
4. revisar métricas;
5. decidir objetivo post-slice;
6. preparar backlog;
7. ejecutar Definition of Ready;
8. abrir un único bloque siguiente.

No se debe empezar automáticamente empleados solo porque aparezcan primero en el roadmap histórico. Debe comprobarse si continúan siendo la mejor inversión de producto.

8. Hitos de producto

Hito	Definición	Estado
H0	proyecto reproduce entorno y build	completado
H1	caminar y construir local válido	completado
H2	inventario y recepción sin pérdida	completado

Hito	Definición	Estado
H3	primer cliente compra	completado
H4	primer día completo	completado
H5	semana guardada/cargada	completado
H6	vertical slice aprobado	pendiente
H7	MVP profundo de tienda	no abierto
H8	negocio multicanal online	visión
H9	ecosistema editorial y desarrollo	visión
H10	plataforma, infraestructura y condición de victoria	visión
H11	Alpha	no planificado
H12	Beta	no planificado
H13	Release Candidate	no planificado
H14	lanzamiento PC/Steam	no planificado

8.1. Regla de madurez de hitos

Un hito solo se marca como completado cuando:

- existe evidencia;
- el resultado es reproducible;
- no depende de una working copy no registrada;
- sus criterios están cerrados;
- su documentación está actualizada;
- existe una decisión de aceptación.

9. Roadmap posterior al vertical slice

9.1. Naturaleza provisional

Las fases siguientes son una **secuencia recomendada**, no una promesa.

No reciben fechas ni números de sprint definitivos hasta completar H6 y revisar:

- diversión;
- retención;
- coste de producción;
- deuda;
- rendimiento;
- respuesta de playtest;
- capacidad de contenido;
- viabilidad comercial.

9.2. Bloque PVS-0 — Freeze y revisión de producto

Objetivo: convertir la experiencia del vertical slice en conocimiento de planificación.

Entregables:

- baseline congelada;
- retrospectiva;
- análisis de métricas;
- lista de fricción;
- deuda priorizada;
- decisiones de alcance;
- backlog post-slice;
- hipótesis de producto;
- roadmap revisado.

Fuera de alcance:

- nuevos sistemas mayores;
- expansión de contenido sin conclusión;
- marketing de características no aprobadas.

Gate de salida:

- siguiente bloque seleccionado;
- Definition of Ready cumplida;
- coste y riesgo razonables;
- criterios de éxito definidos.

9.3. MVP-1 — Empleados

Resultado de producto

Reducir carga operativa mediante agentes visibles sin eliminar el valor de las tareas manuales.

Secuencia recomendada

MVP-1A — Fundación de empleados

- definiciones;
- IDs;
- contratación;
- salario;
- presencia en mundo;
- horarios;
- estado básico;
- guardado.

MVP-1B — Tareas y asignación

- cola de tareas;
- prioridades;
- roles;
- desplazamiento;
- ejecución;

- bloqueo y fallback.

MVP-1C — Fatiga, rendimiento y gestión

- cansancio;
- descansos;
- habilidad;
- feedback;
- costes;
- resultados;
- despido.

Dependencias

- tareas manuales estables;
- navegación fiable;
- economía;
- jornada;
- persistencia;
- UX de prioridades;
- datos configurables.

Gate

- el jugador comprende por qué un empleado actúa o no;
- la automatización no trivializa la tienda;
- no existen dobles mutaciones;
- cierre y save resuelven tareas;
- el rendimiento soporta agentes adicionales.

9.4. MVP-2 — Investigación y ampliaciones

Resultado de producto

Introducir progresión estructurada que desbloquee capacidad, comodidad y expansión.

Secuencia recomendada

- catálogo de nodos;
- requisitos;
- costes;
- progreso;
- desbloques;
- ampliaciones;
- persistencia;
- UX;
- balance.

Reglas

- cada nodo desbloquea una capacidad observable;
- no usar porcentajes abstractos sin representación;
- no bloquear contenido esencial detrás de grind;
- separar research de mejoras puramente cosméticas;
- mantener compatibilidad de save.

Gate

- progresión comprensible;
- decisiones mutuamente significativas;
- efectos verificables;
- no se rompe el balance de la tienda base.

9.5. MVP-3 — Puestos informáticos

Resultado de producto

Añadir un servicio presencial conectado con clientes, espacio, componentes y economía.

Secuencia recomendada

MVP-3A — Setup físico

- muebles;
- componentes;
- tiers;
- montaje;
- ocupación;
- mantenimiento mínimo.

MVP-3B — Servicio

- tarifa;
- duración;
- reserva;
- cliente;
- uso;
- pago;
- cierre.

MVP-3C — Integración

- empleados;
- investigación;
- demanda;
- satisfacción;
- economía;
- resultados;
- persistencia.

Gate

- reutiliza clientes y reservas;
- no duplica economía;
- no bloquea circulación;
- la tarifa es comprensible;

- el estado del puesto es visible;
- la jornada puede cerrar con sesiones activas.

9.6. H7 — MVP profundo de tienda

H7 exige:

- tienda base estable;
- empleados;
- investigación;
- puestos;
- progresión coherente;
- contenido suficiente;
- persistencia;
- balance;
- rendimiento;
- UX;
- build.

Solo después de H7 debe evaluarse la gran expansión multicanal.

9.7. EXP-1 — Comercio online y logística

Resultado de producto

Extender el mismo negocio hacia pedidos online sin crear un inventario paralelo incoherente.

Workstreams:

- catálogo online;
- publicación;
- pedido;
- reserva;
- picking;
- packing;
- embalaje;

- transporte;
- incidencias;
- devoluciones;
- reputación;
- capacidad logística;
- empleados;
- resultados.

Dependencias:

- inventario robusto;
- empleados;
- economía;
- persistencia;
- UI de operaciones;
- capacidad de contenido;
- rendimiento.

Gate H8:

- inventario físico y online consistente;
- pedidos trazables;
- logística visible;
- incidencias recuperables;
- economía integrada;
- jornada o calendario ampliado coherente.

9.8. EXP-2 — Publishing

Resultado:

- propuestas;
- evaluación;
- contratos;
- hitos;

- financiación;
- intervención;
- lanzamiento;
- liquidación.

Debe modelar incertidumbre y capacidad editorial sin convertirse en un simulador puramente tabular.

Gate:

- contratos persistentes;
- riesgo comprensible;
- hitos verificables;
- relación con reputación, capital y mercado;
- ningún pago se duplica;
- los resultados afectan al ecosistema.

9.9. EXP-3 — Desarrollo interno

Resultado:

- brief;
- prototipo;
- playtest;
- greenlight;
- producción;
- hitos;
- build;
- lanzamiento.

Regla de identidad:

El desarrollo interno debe surgir del conocimiento comercial acumulado. No debe reemplazar la tienda ni convertir el juego en un clon de un tycoon de desarrollo.

Gate H9:

- publishing y desarrollo se conectan;

- el mercado aporta información parcial;
- los proyectos tienen coste y capacidad;
- la tienda y el comercio siguen siendo relevantes.

9.10. LATE-1 — Plataforma digital

- catálogo digital;
- usuarios;
- adquisición;
- operación;
- reputación;
- moderación;
- monetización;
- contratos;
- servicio.

9.11. LATE-2 — Infraestructura y servicios

- pools de capacidad;
- módulos;
- mantenimiento;
- incidentes;
- degradación;
- prioridades;
- costes;
- servicios internos y externos.

No debe convertirse en una simulación técnica exhaustiva de centros de datos.

9.12. LATE-3 — Mercado y competidores

- tendencias;
- noticias;
- empresas rivales;

- reacciones;
- oportunidades;
- presión;
- información imperfecta;
- condiciones de victoria.

Gate H10:

- plataforma e infraestructura producen decisiones;
- los competidores son comprensibles;
- no existe omnisciencia;
- no hay bolas de nieve inevitables;
- el postgame conserva objetivos.

10. Preparación de sprints post-H6

10.1. No asignar números demasiado pronto

Los identificadores futuros pueden mantenerse como `MVP-1A`, `MVP-1B`, etc. hasta aprobar:

- prioridad;
- capacidad;
- estimación;
- dependencias;
- versión objetivo.

Después podrán mapearse a Sprints 18, 19, 20 o a otra secuencia sin que el documento finja un compromiso inexistente.

10.2. Tamaño de sprint recomendado

Un sprint debe cerrar:

- una capacidad vertical;

- una transición de estado;
- una integración;
- un gate;
- una deuda delimitada.

Debe dividirse si requiere simultáneamente:

- nueva arquitectura;
- varios modelos persistentes;
- UI completa;
- assets finales;
- múltiples agentes;
- balance;
- build;
- migración de datos.

10.3. Spikes

Un spike:

- tiene pregunta concreta;
- tiene límite;
- no se integra automáticamente;
- produce conclusión;
- identifica riesgo;
- puede terminar recomendando no implementar.

Ejemplos:

- validar navegación con muchos empleados;
- medir coste de instancias visibles;
- probar un modelo de cola online;
- evaluar una librería;
- validar una migración de schema.

11. Dependencias de producción

11.1. Mapa general

Enfoque

→ GDD

→ Vertical Slice Specification

→ TDD

→ Modelo de Datos

→ UX Flow

→ Roadmap

→ Sprint Charter

→ Implementación

→ QA

→ Build

→ Evidencia

→ Cierre

11.2. Dependencias por disciplina

Disciplina	Necesita	Produce
Diseño	enfoque, feedback, métricas	reglas y aceptación
Código	reglas, datos, arquitectura	capacidades
Datos	IDs, ownership, schema	configuración y estado
Escena	arquitectura y referencias	composición jugable
Arte	target, escala, pipeline	assets representativos
UI/UX	estados y acciones	recorridos y feedback
Audio/VFX	eventos y prioridades	feedback
QA	criterios y build	evidencia y defectos
Producción	capacidad, riesgos, estado	orden y gates
Documentación	decisiones y evidencia	continuidad

11.3. Dependencias bloqueantes

Una dependencia es bloqueante si:

- impide probar;
- impide guardar;
- impide cargar;
- impide build;
- invalida una referencia;
- cambia ownership;
- altera una invariante;
- impide aceptación visual;
- requiere una decisión no tomada.

Debe registrarse en el sprint con:

- owner;
- fecha de detección;
- impacto;
- workaround;
- condición de desbloqueo.

12. Backlog y prioridad

12.1. Clases

Prioridad	Significado
P0	bloquea sprint, datos, build o Golden Path
P1	necesario para gate próximo
P2	mejora importante post-gate
P3	expansión o profundidad futura

Prioridad	Significado
P4	idea, polish o experimento

12.2. Regla de entrada

- P0 puede interrumpir.
- P1 entra en planificación del gate.
- P2 no desplaza P0/P1.
- P3 requiere fase aprobada.
- P4 permanece en idea/backlog.

12.3. Backlog no es roadmap

El backlog puede contener muchas ideas. El roadmap solo contiene bloques con:

- propósito;
- dependencias;
- criterio de entrada;
- criterio de salida;
- relación con hitos.

13. Definition of Ready

Un sprint está **READY** cuando:

1. el objetivo puede expresarse en una frase;
2. el problema está demostrado;
3. el alcance está enumerado;
4. las exclusiones están enumeradas;
5. las dependencias están disponibles;
6. las decisiones de diseño están tomadas;

7. los datos necesarios están definidos;
8. la arquitectura objetivo está acordada;
9. los criterios de aceptación existen;
10. el test plan existe;
11. la versión objetivo está definida;
12. la estrategia de save/migración está definida;
13. los assets necesarios están disponibles o sustituidos por representativos aprobados;
14. los riesgos principales están registrados;
15. la evidencia requerida está definida;
16. el sprint es cerrable con la capacidad disponible;
17. no compite con otro sprint principal;
18. existe una condición clara de aborto o reducción.

Un sprint no está listo si su primer objetivo es “averiguar qué hay que hacer”. En ese caso se abre un spike.

14. Definition of Done de sprint

Un sprint se considera cerrado cuando:

1. el entregable existe;
2. los criterios de aceptación pasan;
3. no existen S0/S1;
4. los S2 restantes están aceptados o planificados;
5. las pruebas relevantes pasan;
6. la integración manual pasa;
7. el Golden Path pasa si está afectado;
8. la persistencia se ha validado;
9. el rendimiento se ha revisado cuando procede;
10. la accesibilidad se ha revisado cuando procede;
11. la build se ha generado cuando el gate lo exige;
12. `Player.Log` se ha revisado;

13. los defectos están clasificados;
14. la documentación está actualizada;
15. la trazabilidad está actualizada;
16. la versión y SHA están registrados;
17. las evidencias están archivadas;
18. el resultado puede reproducirse;
19. se celebra revisión;
20. existe una decisión de cierre.

“Funciona en mi Editor” no constituye Definition of Done.

15. Ciclo de vida de un sprint

15.1. Preparación

- revisar baseline;
- seleccionar objetivo;
- congelar alcance;
- crear charter;
- crear criterios;
- crear test plan;
- identificar riesgos;
- definir versión;
- crear rama o estrategia de integración según política.

15.2. Ejecución

- implementar por slices;
- mantener compilación;
- ejecutar pruebas frecuentes;
- registrar decisiones;

- evitar cambios laterales;
- demostrar resultados parciales;
- actualizar defectos.

15.3. Integración

- conectar escena;
- conectar datos;
- conectar UI;
- conectar persistencia;
- verificar input;
- verificar lifecycle;
- ejecutar regresión.

15.4. Validación

- acceptance matrix;
- QA manual;
- suites;
- Golden Path;
- rendimiento;
- build;
- logs;
- accesibilidad;
- localización.

15.5. Cierre

- corregir o aceptar deuda;
- actualizar docs;
- archivar evidencia;
- registrar versión y SHA;
- revisar objetivo;

- cerrar;
- no añadir “una última mejora” después de la aceptación sin reabrir formalmente.

15.6. Handoff

Debe permitir continuar sin memoria del chat.

Incluye:

- estado;
- último SHA validado;
- working copy;
- pruebas;
- bloqueos;
- siguiente acción;
- qué no hacer;
- documentación prioritaria.

16. Gestión de alcance

16.1. Cambio menor

Puede gestionarse dentro del sprint si:

- no añade una capacidad;
- no altera persistencia;
- no cambia arquitectura;
- no cambia un gate;
- no aumenta riesgo de forma material;
- cabe dentro del esfuerzo reservado.

16.2. Cambio mayor

Requiere decisión formal si:

- añade sistema;
- cambia ownership;
- modifica schema;
- altera escena de producción;
- cambia plataforma;
- cambia build;
- adelanta contenido diferido;
- elimina un criterio;
- redefine el hito;
- compromete fecha externa.

16.3. Plantilla de cambio

- descripción;
- motivo;
- evidencia;
- alternativas;
- impacto;
- riesgo;
- trabajo retirado;
- documentos afectados;
- decisión;
- fecha;
- responsable.

16.4. Protección contra sprints indefinidos

Si un sprint no converge:

1. comprobar si el objetivo es demasiado grande;

2. aislar la parte demostrable;
3. crear spike para incertidumbre;
4. retirar contenido no esencial;
5. registrar bloqueo;
6. cerrar con deuda aceptada solo si el gate no queda comprometido;
7. cancelar y replantear si la hipótesis es inválida.

No se debe extender silenciosamente un sprint durante múltiples ciclos sin redefinirlo.

17. QA dentro del roadmap

17.1. Pirámide

- dominio: pruebas unitarias;
- aplicación: servicios e integración;
- infraestructura: serialización, archivos, input;
- presentación: PlayMode;
- escena: integración;
- build: Golden Path;
- usuario: playtest.

17.2. Severidades

Severidad	Significado
S0	corrupción, pérdida grave o bloqueo total
S1	Golden Path o capacidad principal imposible
S2	función importante degradada
S3	fricción significativa
S4	defecto menor

17.3. Gate

- S0: 0;
- S1: 0;
- S2: cerrados o aceptados formalmente;
- S3/S4: clasificados.

17.4. Regresión heredada

Cada sprint posterior debe proteger sistemas cerrados. Un test nuevo no sustituye a los anteriores salvo que la regla haya sido formalmente sustituida.

18. Build y versionado

18.1. Versiones

Cada sprint que cambie capacidad jugable debe:

- proponer versión;
- actualizar versión al cerrar;
- registrar SHA;
- registrar schema si cambia;
- registrar build si existe.

18.2. Tipos de build

Tipo	Uso
desarrollo	diagnóstico interno
QA	pruebas y Golden Path
candidato de hito	aceptación formal
demo	distribución controlada

Tipo	Uso
Alpha	cobertura funcional
Beta	contenido y estabilización
RC	candidato a lanzamiento
Release	distribución pública

18.3. Registro mínimo

- nombre;
- versión;
- SHA;
- fecha;
- plataforma;
- configuración;
- escenas;
- resultado;
- tests;
- hardware;
- known issues;
- checksum cuando proceda.

19. Evidencia obligatoria

19.1. Por sprint

- charter;
- acceptance matrix;
- test plan;
- resultados;

- capturas;
- logs;
- build record;
- cambios;
- defectos;
- cierre;
- handoff.

19.2. Por hito

- build;
- vídeo o recorrido;
- rendimiento;
- matriz completa;
- known issues;
- versión;
- SHA;
- baseline documental;
- decisión de aprobación.

19.3. Calidad de evidencia

La evidencia debe ser:

- reproducible;
 - fechada;
 - asociada a versión y SHA;
 - legible;
 - suficiente;
 - no dependiente de mensajes privados;
 - guardada en ubicación estable.
-

20. Riesgos de producción

Riesgo	Probabilidad	Impacto	Mitigación	Gate
scope creep	alta	alto	alcance congelado y gates	todos
escena representativa rompe lógica	media	alto	autoría explícita y fallback	S16
assets con escala/ejes incorrectos	media	medio	prefabs, revisión e importación controlada	S16
UI y mundo procesan mismo clic	alta	alto	contextos y EventSystem	S16
build diverge del Editor	media	alto	build frecuente y logs	S16/S17
save pierde estado nuevo	media	crítico	contratos, schema y recovery	todos
rendimiento de agentes	media	alto	profiling y límites	S17/MVP
demasiada UI	alta	medio	revelado progresivo y playtest	MVP/EXP
automatización trivializa juego	media	alto	tareas manuales primero	empleados
roadmap futuro se interpreta como promesa	media	alto	estados provisionales	comercial
documentación se desincroniza	media	alto	DoD y baseline	todos
desarrollo individual se sobrecarga	alta	alto	un sprint principal y contenido mínimo	todos
deuda procedural permanece indefinida	media	medio	plan de retirada tras migración	S16/PVS-0
migraciones de schema se acumulan	media	alto	convergencia y pruebas	post-VS
contenido consume tiempo antes de validar	alta	medio	contenido representativo	todos

20.1. Registro de riesgo

Cada riesgo debe incluir:

- trigger;
- owner;
- probabilidad;
- impacto;
- mitigación;
- contingencia;
- estado;
- fecha de revisión.

21. Métricas de producción

21.1. Flujo

- sprints cerrados;
- tiempo de ciclo;
- trabajo retirado;
- defectos reabiertos;
- bloqueos;
- deuda creada y pagada;
- estabilidad de alcance.

21.2. Calidad

- pruebas;
- tasa de PASS;
- S0-S4;
- regresiones;
- fallos de build;
- fallos de guardado;
- errores en logs.

21.3. Producto

- Golden Path completado;
- tiempo hasta primera venta;
- finalización de jornada;
- abandono;
- rendimiento;
- comprensión;
- accesibilidad.

21.4. Interpretación

Las métricas deben orientar decisiones, no premiar volumen.

No usar:

- líneas de código;
- commits;
- archivos creados;
- número de tareas cerradas

como medida principal de progreso.

22. Revisión del roadmap

22.1. Revisión semanal

Comprobar:

- plan vs realizado;
- objetivo;
- bloqueos;
- defectos;
- riesgos;
- deuda;
- tests;
- build;
- documentación;
- capacidad;
- siguiente acción.

22.2. Revisión por sprint

- ¿se cerró el resultado?
- ¿qué estimación falló?
- ¿qué dependencia no estaba lista?
- ¿qué deuda se creó?
- ¿qué debe cambiar en DoR?
- ¿qué evidencia faltó?
- ¿el roadmap sigue siendo válido?

22.3. Revisión por hito

- visión;
- alcance;
- calidad;
- coste;
- respuesta de playtest;
- viabilidad comercial;
- orden de fases;
- características que deben eliminarse;
- riesgo de continuar.

22.4. Reestimación

Reestimar no es fracaso. Ocultar desviaciones sí lo es.

Toda reestimación debe:

- usar evidencia;
- conservar historia;
- explicar cambio;
- no convertir una idea en compromiso;
- actualizar dependencias;
- actualizar documentos afectados.

23. Hitos comerciales

23.1. Vertical Slice

Objetivo interno de validación.

No implica distribución pública.

23.2. MVP de tienda

Debe demostrar profundidad suficiente con empleados, investigación y puestos, si esas líneas se mantienen después de H6.

23.3. Demo pública

Solo se abre cuando:

- el build es estable;
- la experiencia representa el producto;
- no contiene promesas engañosas;
- la localización está revisada;
- existe telemetría o método de feedback;
- existe soporte mínimo;
- el alcance posterior está controlado.

23.4. Alpha

Criterio:

- todas las capacidades comprometidas existen;
- el contenido puede estar incompleto;
- se pueden completar recorridos principales;
- persistencia y migraciones funcionan;
- no se añaden sistemas mayores sin cambio formal.

23.5. Beta

Criterio:

- feature complete;
- contenido casi completo;
- balance y rendimiento en foco;
- compatibilidad;
- accesibilidad;
- localización;
- Steam;
- soporte.

23.6. Release Candidate

- sin S0/S1;
- S2 aceptados;
- regresión;
- save;
- actualización;
- build;
- store assets;
- legal;
- créditos;
- privacidad;
- logs;
- rollback.

23.7. Lanzamiento

- RC aprobada;
- publicación;
- monitorización;
- canal de incidencias;

- hotfix plan;
 - soporte;
 - backup;
 - roadmap postlaunch separado.
-

24. Elementos no comprometidos

No están comprometidos por este roadmap:

- fechas públicas;
- multijugador;
- workshop;
- modding;
- versiones de consola;
- servicios online reales;
- plataforma completa en primera versión;
- competidores exhaustivos;
- contenido masivo;
- arte final para todos los sistemas;
- Steam Cloud antes de aprobación;
- logros antes de aprobación;
- demo antes de gate;
- todas las líneas empresariales en el lanzamiento inicial.

Cualquier incorporación requiere análisis de coste, valor, riesgo y capacidad.

25. Secuencia inmediata recomendada

1. Confirmar working copy y SHA real.
2. Mantener Sprint 16 como único sprint activo.
3. Ejecutar los 11 pasos de StoreInitial.
4. Obtener aprobación visual manual.
5. Ejecutar Golden Path post-integración.
6. Generar build post-integración.
7. Revisar Player.log.
8. Cerrar Sprint 16 documentalmente.
9. Abrir Sprint 17.
10. Ejecutar balance, profiling, persistencia, UX y QA.
11. Generar build candidata de H6.
12. Celebrar gate H6.
13. Congelar baseline.
14. Ejecutar PVS-0.
15. Elegir un único bloque post-slice.

26. Matriz de decisiones de fase

Pregunta	Sí	No
¿Sprint 16 tiene aprobación visual y build?	abrir S17	continuar S16
¿S17 cumple DoD?	evaluar H6	continuar estabilización
¿H6 está aprobado?	congelar baseline	corregir o reducir
¿Hay evidencia de que empleados son siguiente prioridad?	preparar MVP-1	evaluar otra opción
¿MVP de tienda está aprobado?	evaluar online	profundizar o estabilizar
¿Online está validado?	evaluar publishing	no abrir EXP-2
¿Publishing aporta valor probado?	evaluar desarrollo	no abrir EXP-3
¿La escala soporta plataforma?	discovery LATE	mantener alcance

27. Plantilla de Sprint Charter

```
# Sprint XX – Nombre

## Estado
PROPOSED / READY / IN PROGRESS / CLOSED

## Versión objetivo

## Problema

## Objetivo

## Alcance incluido

## Fuera de alcance

## Dependencias

## Riesgos

## Entregables

## Criterios de aceptación

## Test plan

## Persistencia y migración

## Rendimiento

## Accesibilidad

## Build

## Evidencias

## Definition of Done

## Condición de aborto o reducción
```

28. Plantilla de cierre

```
# Sprint XX – Closure Record
```

```
## Resultado

## Versión y SHA

## Alcance entregado

## Criterios

## Pruebas

## Validación manual

## Build y Player.log

## Defectos

## Deuda aceptada

## Documentación

## Evidencias

## Decisión
PASS / PASS WITH ACCEPTED DEBT / FAIL

## Siguiendo gate
```

29. Regla de mantenimiento del roadmap

Este archivo debe actualizarse cuando:

- se abre o cierra un sprint;
- cambia el estado de un hito;
- se aprueba una deuda;
- cambia una dependencia;
- se modifica el alcance;
- se genera una baseline;
- un playtest cambia prioridad;
- una fase se cancela;
- se compromete una fecha externa.

No debe reescribirse el histórico para aparentar que el plan siempre fue correcto. Las decisiones sustituidas deben conservarse en trazabilidad.

Anexo A. Trazabilidad de fuentes

Fuente	Líneas	Palabras aprox.	SHA-256
Roadmap v0.3	150	870	93b345037c1356c7e176a7377bd66a168aa356366aa3c4e16062b09fcb46ee60
Roadmap v0.4	174	988	e9976daa630e8bc1460831eea0c3d280d24f38d3907ef3dd97baec6e35661404
Roadmap v0.5	96	523	d0623a03b2ff75fa95ecd6bbaa5fcbaf0fa8faada31fc2a6b4a569caa50f0e285
Roadmap v0.6	90	469	f7b50c87cbc0faa2478f677f868e2e29dd1e74abfe74c1cbb30aed74ab7ac70a
Current Project Baseline Record	19	73	98e04d98696a645237fcf80cbca55467a16a09e6bd0d46df1a74637a222ca2e4
Sprint History Summary	22	179	db3967d73cb90a9879c223328a922431bf3685b1a8e57afa190511403dc9d1eb
Sprints 00-15 Consolidated Closure Ledger	52	551	17c317dacaec6438f7a1180b6b2811184a0d995fb41964ed2fe9ab46e1d35aab
Sprint 16 Current Status	20	88	bc336c8bf852d942d1b0289cebfdde9db889088990fcc6151df5db559e94d1a9
Sprint 17 Opening Brief	3	36	fb2c1a557e3b6cd765497315fa554eacc048c2fd21a2be9c7f5f28895a189ac1
Current Project Handoff	37	161	7a297d84a36ecdd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87
ADR-0035 StoreInitial Manual Scene Authoring	20	106	0e69d8b664239477738f3dcce96eb5cfc2ff6e3db8817261f639ac4d64092b3
StoreInitial Authoring Plan	26	88	f67d195bcda2bd1f861dcf5c53d0a299260c1f04c2f299250856a76616131eba
00_Enfoque_y_Alcance.md	4495	18925	63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f
01_Game_Design_Document.md	4490	19760	a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177
02_Vertical_Slice_Specification.md	1505	9834	75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f
03_Technical_Design_Document.md	3305	13774	66264703c5fff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
04_Modelo_de_Datos.md	2691	12944	d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4
05_UX_Flow.md	2458	8720	e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e

A.1. Uso de las fuentes

- Roadmap v0.6: estado global y alcance de S16/S17.
- Sprint 16 Current Status: trabajo aceptado y pendiente.
- StoreInitial Authoring Plan: secuencia de 11 pasos.
- ADR-0035: decisión de autoría manual y consecuencias.
- Sprint 17 Opening Brief: gate de apertura y prohibición de sistemas nuevos.
- Closure Ledger: verdad histórica 0–15.
- Baseline Record: tests, versión y SHAs documentales.
- Handoff: continuidad, restricciones y siguiente acción.
- Roadmaps v0.3–v0.5: fases, hitos, prioridades y evolución histórica.
- Documentos 00–05: autoridad de producto, diseño, técnica, datos y UX.

Anexo B. Decisiones sustituidas

Materia	Decisión histórica	Regla vigente
Estado	preproducción desde cero	S0–15 cerrados, S16 activo
Tienda	10 × 10 celdas / 5 × 5 m	10 × 15 m , grid 20 × 30
S16	arte y audio genéricos	autoría y conexión de StoreInitial
S17	estabilización genérica	gate detallado, sin sistemas nuevos
H6	objetivo abstracto	aprobación formal con evidencia
Post-VS	secuencia asumida	roadmap provisional sujeto a PVS-0
Fechas	estimaciones implícitas	no se asignan sin capacidad
Build	una tarea más	evidencia obligatoria de gate
Documentación	apoyo	parte de DoD
Deuda	implícita	registrada, aceptada y planificada

Anexo C. Glosario de producción

Término	Definición
Baseline	conjunto identificado de código, datos, build, pruebas y documentos
Build candidata	build sometida a aceptación de hito
Capacidad	resultado jugable o técnico observable
Charter	contrato de apertura del sprint
DoR	condiciones para poder empezar
DoD	condiciones para poder cerrar
Gate	decisión formal que permite avanzar
Golden Path	recorrido integrado obligatorio
Handoff	información suficiente para continuar
Hito	resultado de producto aceptado
Spike	investigación limitada para resolver una pregunta
Working copy	estado local todavía no publicado o identificado
Deuda aceptada	desviación registrada que no bloquea el gate
Regresión	pérdida de una capacidad previamente aceptada

Anexo D. Ruta de almacenamiento

```
Documentation/  
└─ 06_Production_Roadmap_y_Sprint_Plan.md
```

Estado del documento: fuente vigente de planificación y producción para la nueva carpeta `Documentation/`. Su estado operativo deberá actualizarse al cerrar Sprint 16, abrir Sprint 17 y decidir el gate H6.

Fin del documento.

Fuente: 06_Production_Roadmap_y_Sprint_Plan.md · SHA-256: `d00b156c0ad1c66069278119c55f76c738a577a3e2835f770208d1a5f2faadff`

Diseño PDF: paleta VRM Games definida en la documentación de Cartridge & Cloud.